

API reference

Everything the Android client talks to, where each call is defined on both sides, and how to run against it.

42 endpoints across 33 route files — that is the entire server surface of `main`; the app uses all of it. The inventory below is generated from the Retrofit interfaces, not written from memory, so it matches the shipped code.

The tables list 43 rows for 42 endpoints: `GET /api/businesses` appears twice because two features call it independently — the dashboard for portfolio totals, Business Check for its list.

How to run against it

The client never invents a base URL. It comes from `BuildConfig.API_BASE_URL`, set per product flavor in `app/build.gradle.kts`:

FLAVOR	BASE URL	NEEDS A SERVER
<code>mock</code>	unused	no — in-memory fixtures
<code>live</code>	<code>http://10.0.2.2:3000/</code>	yes

`10.0.2.2` is how the Android emulator reaches the host's loopback. On a physical device, change it to your machine's LAN address and add that host to `app/src/main/res/xml/network_security_config.xml` — cleartext is allowed only for loopback hosts, never the open internet.

Start the server:

```
git clone https://github.com/L11cif3r/ChaanBean.git && cd ChaanBean
npm install
npx prisma db push
npm run db:seed
npm run dev
```

Then install the client:

```
./gradlew :app:installLiveDebug # against the server
./gradlew :app:installMockDebug # standalone, no server
```

If port 3000 is taken, run the server on another port (`npx next dev -p 3001`) and change `API_BASE_URL` to match — the port is baked into the APK at build time, so the app must be rebuilt after changing it.

WATCHING THE TRAFFIC

Debug builds log full request and response bodies through an OkHttp interceptor (`core/network/Network.kt`); release builds log only the status line.

```
adb logcat | grep OkHttpClient
```

No authentication

`main` performs none, so the client sends none. There is no token, header, cookie or session on any of the 42 calls — `POST /api/auth` returns a user object without checking a credential, and no other endpoint reads one. Anything reachable by the app is reachable by anyone who can reach the server.

The client keeps identity in one file, `core/di/SessionStore.kt`, so adding real auth means changing that file plus one OkHttpClient rather than 42 call sites.

How a call is wired

Each feature owns a vertical slice, so a new endpoint touches one package:

<code>feature/<area>/data/<Area>Api.kt</code>	Retrofit interface – the HTTP contract
<code>feature/<area>/data/<Area>Models.kt</code>	@Serializable request/response types
<code>feature/<area>/data/<Area>Repository.kt</code>	interface + Live + Mock + Module factory

The repository interface has two implementations, chosen at runtime by the flavor flag:

```
object VendorsModule {
    fun repository(container: AppContainer): VendorsRepository =
        if (container.useMock) MockVendorsRepository()
        else LiveVendorsRepository(container.retrofit.create(VendorsApi::class.java))
}
```

Every repository method returns `Outcome<T>` — `Loading`, `Ok(value)` or `Err(message)` — and screens render the three states through `OutcomeContent`.

Two conventions matter when adding a call:

- **Retrofit paths carry no leading slash.** The base URL already ends in one.
- **Every @Serializable field needs a default.** `main`'s handlers omit fields freely, so a field without a default throws at parse time on a perfectly valid response.

The 42 endpoints

Server column is the file under `src/app/` on branch `main`; client column is the file under `app/src/main/kotlin/com/chaanbean/mobile/feature/` here.

IDENTITY

METHOD	PATH	SERVER	CLIENT
POST	<code>/api/auth</code>	<code>api/auth/route.ts</code>	<code>auth/data/AuthApi.kt</code>
GET	<code>/api/admin/auth</code>	<code>api/admin/auth/route.ts</code>	<code>admincore/data/AdminApi.kt</code>
POST	<code>/api/admin/auth</code>	<code>api/admin/auth/route.ts</code>	<code>admincore/data/AdminApi.kt</code>

`POST /api/auth` is four operations behind one path, selected by an `action` field: `login_client`, `register_client`, `login_admin`, `register_admin`.

```
{ "action": "login_client", "email": "ops@acme.in", "companyName": "Acme" }
```

`action` deliberately has **no Kotlin default**: the shared `Json` uses `encodeDefaults = false`, so a defaulted value would be dropped from the body and the server would answer `Invalid action.`

DASHBOARD AND HEALTH

METHOD	PATH	SERVER	CLIENT
GET	/api/health	api/health/route.ts	dashboard/data/DashboardApi.kt
GET	/api/buyers	api/buyers/route.ts	dashboard/data/DashboardApi.kt
GET	/api/recovery	api/recovery/route.ts	dashboard/data/DashboardApi.kt
GET	/api/businesses	api/businesses/route.ts	dashboard/data/DashboardApi.kt

Only `checks.database` in the health response is a real probe. `verificationGateway`, `policyEngine`, `riskScoringEngine` and `voiceSystem` are hardcoded `"operational"` literals that probe nothing.

BUSINESS CHECK — THE LARGEST SURFACE

METHOD	PATH	CLIENT
GET	/api/businesses	businesscheck/data/BusinessCheckApi.kt
POST	/api/businesses	businesscheck/data/BusinessCheckApi.kt
GET	/api/businesses/search	businesscheck/data/BusinessCheckApi.kt
POST	/api/businesses/compare	businesscheck/data/BusinessCheckApi.kt
GET	/api/businesses/{id}	businesscheck/data/BusinessCheckApi.kt
PATCH	/api/businesses/{id}	businesscheck/data/BusinessCheckApi.kt
GET	/api/businesses/{id}/risk	businesscheck/data/BusinessCheckApi.kt
GET	/api/businesses/{id}/credit	businesscheck/data/BusinessCheckApi.kt
GET	/api/businesses/{id}/audit	businesscheck/data/BusinessCheckApi.kt
POST	/api/businesses/{id}/verify	businesscheck/data/BusinessCheckApi.kt
POST	/api/businesses/{id}/manual-verify	businesscheck/data/BusinessCheckApi.kt
GET	/api/businesses/{id}/documents	businesscheck/data/BusinessCheckApi.kt
POST	/api/businesses/{id}/documents	businesscheck/data/BusinessCheckApi.kt
GET	/api/businesses/{id}/documents/{docId}	businesscheck/data/BusinessCheckApi.kt

Server files live under `src/app/api/businesses/`.

`manual-verify` is one path with four payload shapes — MCA, GST, Udyam and eCourts — so the client exposes four typed functions (`manualVerifyMca`, `manualVerifyGst`, `manualVerifyUdyam`, `manualVerifyCourtCases`) rather than one untyped blob.

Document upload is the only `multipart/form-data` call; the handler reads `file`, `category` and `fiscalYear`.

Two behaviours worth knowing before you build on these:

- `GET /api/businesses/{id}/documents/{docId}` ignores the business id and looks the document up by `docId` alone, so a document from any business is reachable through any business's path.
- The list response omits relations (`identifiers`, `riskSignals`, `courtCases`, ...) and carries `_count` instead; the detail response carries the relations but **not** `_count`. The client falls back to relation sizes on detail.

DEBTORS, RECOVERY AND ARBITRATION

METHOD	PATH	SERVER	CLIENT
POST	/api/buyers	api/buyers/route.ts	debtors/data/DebtorsApi.kt
POST	/api/risk	api/risk/route.ts	debtors/data/DebtorsApi.kt
POST	/api/recovery	api/recovery/route.ts	recovery/data/RecoveryApi.kt
POST	/api/recovery/settle	api/recovery/settle/route.ts	recovery/data/RecoveryApi.kt
GET	/api/audio/{hash}	api/audio/[hash]/route.ts	recovery/data/RecoveryApi.kt
GET	/api/arbitration	api/arbitration/route.ts	arbitration/data/ArbitrationApi.kt
POST	/api/arbitration	api/arbitration/route.ts	arbitration/data/ArbitrationApi.kt

GET /api/recovery serves two response shapes from one path: the worklist when called bare, and a flattened account with interest and voice script when called with ?creditAccountId=. The client models these as two functions.

POST /api/recovery runs a policy tick:

```
{ "creditAccountId": "cmtrnlkx003cvino4kta90sy" }
```

```
{ "success": true, "result": { "level": "L1", "action": "polite_reminder",  
  "channel": "whatsapp", "ruleId": "POL-L1-001" } }
```

Three cautions carried into the UI:

- /api/recovery/settle is not idempotent. It writes balance, escalation state and evidence log as three separate un-wrapped calls, with no idempotency key and no signature check, and silently discards overpayment via `Math.max(0, ...)`. A replayed request applies twice.
- GET /api/recovery with no creditAccountId calls `findMany` with no `where` clause, returning credit accounts across all companies, while /api/buyers is scoped to one. Single-tenant seed data hides this.
- /api/audio/{hash} returns synthesized tones, not a recording — a chime plus formant tones generated per request, ignoring the hash. Add ?format=json for metadata instead of the WAV body.

TRUST HUB, VENDORS AND VERIFICATION

METHOD	PATH	SERVER	CLIENT
GET	/api/trust-hub/verify	api/trust-hub/verify/route.ts	trusthub/data/TrustHubApi.kt
POST	/api/trust-hub/register	api/trust-hub/register/route.ts	trusthub/data/TrustHubApi.kt
POST	/api/trust-hub/defaults	api/trust-hub/defaults/route.ts	trusthub/data/TrustHubApi.kt
GET	/api/vendors	api/vendors/route.ts	vendors/data/VendorsApi.kt
POST	/api/vendors	api/vendors/route.ts	vendors/data/VendorsApi.kt
POST	/api/verification	api/verification/route.ts	backgroundcheck/data/BackgroundCheckApi.kt
GET	/api/verification/status	api/verification/status/route.ts	backgroundcheck/data/BackgroundCheckApi.kt
POST	/api/verification/otp-initiate	api/verification/otp-initiate/route.ts	backgroundcheck/data/BackgroundCheckApi.kt
POST	/api/verification/otp-verify	api/verification/otp-verify/route.ts	backgroundcheck/data/BackgroundCheckApi.kt
GET	/api/search	api/search/route.ts	search/data/SearchApi.kt

- /api/trust-hub/verify synthesises PAN, GSTIN and CIN from a hash of the company name and returns them as passed checkpoints. They are not verified records.

- `/api/verification/otp-verify` declares an `otp` parameter and never reads it, so any code is accepted.

ADMIN CONSOLE

METHOD	PATH	SERVER	CLIENT
GET	<code>/api/admin/customers</code>	<code>api/admin/customers/route.ts</code>	<code>admincore/data/AdminApi.kt</code>
GET	<code>/api/admin/financials</code>	<code>api/admin/financials/route.ts</code>	<code>admincore/data/AdminApi.kt</code>
GET	<code>/api/admin/marketing</code>	<code>api/admin/marketing/route.ts</code>	<code>admingrowth/data/AdminGrowthApi.kt</code>
GET	<code>/api/admin/pipeline</code>	<code>api/admin/pipeline/route.ts</code>	<code>admingrowth/data/AdminGrowthApi.kt</code>
POST	<code>/api/admin/pipeline</code>	<code>api/admin/pipeline/route.ts</code>	<code>admingrowth/data/AdminGrowthApi.kt</code>

`/api/admin/customers` recomputes health per request from activity, wallet balance and report consumption, and **ignores the stored `Company.healthScore` column** — so the label can disagree with the database.

`/api/admin/financials` takes `?role=owner`; it is the only endpoint that varies by role, and it gates on a value the caller supplies.

Contract tests

The wire format is not assumed. `app/src/test/resources/fixtures/` holds 22 responses captured from a running server, and `ApiContractTest` parses each with the production models.

The load-bearing test walks each model's serialization descriptor against the real payload and reports fields the server never sent — the silent-null bug that compilation cannot catch, where a misnamed field quietly deserializes to a default.

```
./gradlew :app:testMockDebugUnitTest --tests '*ApiContractTest*'
```

Recapture the fixtures after a server change:

```
curl -s http://localhost:3000/api/businesses > app/src/test/resources/fixtures/businesses.json
```

Third-party keys

`main`'s `.env.example` declares thirteen provider keys:

```
APISETU_API_KEY  CIBIL_API_KEY    EXPERIAN_API_KEY  CRIF_API_KEY
WHATSAPP_BSP_API_KEY  SENDGRID_API_KEY  VOBIZ_SIP_ENDPOINT
POLLY_ACCESS_KEY  GEMINI_API_KEY    IT_PORTAL_API_KEY  GST_PORTAL_API_KEY
DATABASE_URL      REDIS_URL
```

Only `DATABASE_URL` is required to run. Five of the keys are read by no code at all, and the whole 21k-line `src/` tree makes exactly one outbound HTTP call — an APIsetu GST turnover lookup. Every other "provider client" resolves from the local database. Telephony, WhatsApp, SMS, e-sign and settlement return hardcoded literals.

So the app needs **one** external dependency to run end to end: the database behind `main`. The other twelve keys are placeholders for integrations that do not exist yet.